Software Engineering

BCSE301L

Module 7

Dr. Naveenkumar Jayakumar
Dept. of Computational Intelligence
PRP-217-4

Software Quality Assurance

- A systematic process of ensuring that software products meet specified requirements and are free from defects.
- It encompasses the entire software development lifecycle, focusing on **improving the process of software development** to prevent errors and ensure the delivery of high-quality software.
- SQA involves monitoring and auditing processes, methodologies, and standards to ensure compliance with established quality benchmarks

Quality Assurance - Goals

- Ensure the software meets functional and non-functional requirements.
- Minimize risks associated with software failure or defects.
- Enhance customer satisfaction by delivering reliable, efficient, and user-friendly software.
- Maintain consistency and repeatability in the software development process.

Quality Assurance – Process Vs Product Metrics

- Categories of measurements used to evaluate and improve the quality of software development and the final software.
- Process metrics focus on the processes involved in software development.
- They measure the efficiency, effectiveness, and quality of the activities and workflows used to develop software.
- The goal is to improve the development process itself

Quality Assurance – Process Vs Product Metrics

- Product metrics focus on the software product itself
- Evaluating its quality, functionality, and performance.
- These metrics help ensure that the final product meets user expectations and adheres to quality standards.

Quality Assurance – Process Vs Product Metrics

Aspect	Process Metrics	Product Metrics
Focus	Development processes and workflows	Final software product and its characteristics
Purpose	Improve efficiency and effectiveness of development	Ensure the product meets quality and performance goals
Examples	Defect density, cycle time, test coverage	Defect count, MTBF, usability, performance metrics
Timeframe	Measured throughout the development lifecycle	Measured primarily after the product is built or released
Impact	Long-term impact on process improvement	Immediate impact on product quality and user experience

Quality Assurance – Process Vs Product Metrics

Purpose of Process Metrics	Purpose of Product Metrics
➢ To identify bottlenecks or inefficiencies in the development process.	➢ To assess the overall quality and reliability of the software product
➢ To track improvements over time as process changes are implemented.	➢ To ensure that the product meets functional and non-functional requirements (e.g., performance, security, usability)
➢ To ensure that best practices are being followed consistently.	➢ To identify areas where the product may need improvement or additional testing
➢ To provide feedback for continuous process improvement	➢ To provide feedback to developers and testers on the quality of the final product.

Quality Assurance – Process Vs Product Metrics

- **Examples of Process Metrics:**
- Defect Density : Number of defects found per unit of code (e.g., per 1,000 lines of code). This metric helps assess the quality of the development process.
- Code Review Efficiency : Percentage of defects identified during code reviews versus those found later in testing. This indicates how effective the review process is at catching issues early.
- Cycle Time : The time it takes to complete a specific phase of the development process (e.g., from requirements gathering to deployment). Shorter cycle times may indicate a more efficient process.

Quality Assurance – Process Vs Product Metrics

- **Examples of Process Metrics:**
- Test Coverage : The percentage of code covered by automated tests. Higher test coverage generally indicates a more thorough testing process.
- Defect Removal Efficiency (DRE) : The percentage of defects found and fixed before release compared to the total number of defects (including those found after release). A high DRE suggests an effective defect detection and removal process.
- Change Failure Rate : The percentage of changes or updates to the software that result in failures or incidents. Lower rates indicate a more stable development process.

Quality Assurance – Process Vs Product Metrics

- **Examples of Product Metrics:**
- Defect Count : The total number of defects found in the software after release. Fewer defects indicate higher product quality.
- Mean Time Between Failures (MTBF) : The average time between system failures. A higher MTBF indicates a more reliable product.
- Mean Time to Repair (MTTR) : The average time it takes to fix a defect or issue once it is discovered. Lower MTTRs suggest faster resolution of problems.

Quality Assurance – Process Vs Product Metrics

- **Examples of Product Metrics:**
- Performance Metrics : Measures such as response time, throughput, and resource utilization. These metrics assess how well the software performs under various conditions.
- Usability Metrics : Metrics like task success rate, error rate, and user satisfaction scores. These evaluate how easy and intuitive the software is to use.
- Code Complexity : Measures such as cyclomatic complexity, which assesses the complexity of the codebase. Simpler code is generally easier to maintain and less prone to errors.

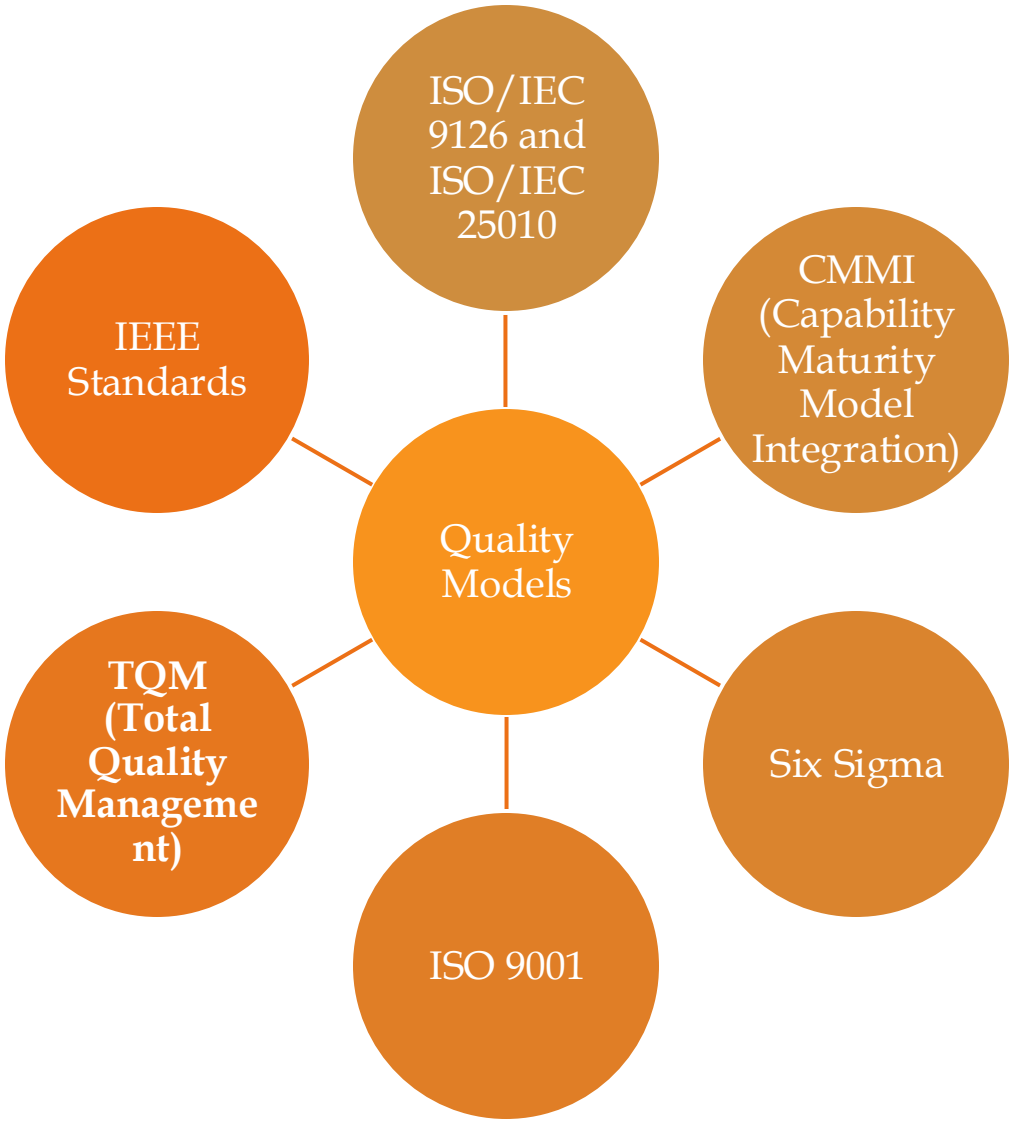
Quality Standard Model

- A structured framework or set of guidelines that defines the criteria, processes, and practices required to ensure the quality of a product, service, or system
- These models provide a benchmark for organizations to measure, evaluate, and improve their processes and outputs, ensuring they meet specific quality standards, regulatory requirements, and customer expectations

Quality Standard Model

- A Quality Standard Model outlines the principles, methodologies, and best practices for developing, testing, and maintaining high-quality software.
- It serves as a reference point for organizations to establish consistent, repeatable processes that lead to reliable and efficient software products.

Quality Standard Models



Quality Standard Models - ISO

- The ISO Quality Model is a framework that defines what "quality" means in software and how to measure it.
- It helps organizations evaluate and improve the quality of their software products by breaking it down into key characteristics.
- The most widely used version is ISO/IEC 25010 , which replaced the older ISO/IEC 9126 .

Quality Standard Models – ISO Core Idea

- The ISO Quality Model defines software quality as a combination of measurable attributes or characteristics.
- These characteristics describe how well the software performs in different areas, such as usability, reliability, and security.

Quality Standard Models – ISO-8 Key Quality Characteristics (ISO/IEC 25010)

Functionality	Performance Efficiency	Compatibility	Usability
• **Does the software do what it's supposed to do?** • Example: A calculator app correctly performs addition, subtraction, etc.	• **How fast and resource-efficient is the software?** • Example: A website loads quickly even with many users.	• **Does the software work well with other systems, devices, or platforms?** • Example: An app works on both Windows and macOS.	• **Is the software easy to use and understand?** • Example: A mobile app has a simple, intuitive interface.

Quality Standard Models – ISO-8 Key Quality Characteristics (ISO/IEC 25010)

Reliability	Security	Maintainability	Portability
• **Does the software work consistently without crashing or failing?** • Example: A banking app stays operational 24/7 without errors.	• **How well does the software protect data and prevent unauthorized access?** • Example: An e-commerce site encrypts user passwords.	• **Is the software easy to update, fix, or modify over time?** • Example: Developers can quickly add new features to the software.	• **Can the software be easily moved or adapted to different environments?** • Example: A game runs smoothly on both Android and iOS devices.

Quality Standard Models – How it Works

- **Measure Quality** : Each characteristic is broken down into sub-characteristics and metrics (e.g., response time for performance efficiency).
- **Improve Software** : Teams use these characteristics to identify weaknesses and make improvements.
- **Ensure Consistency** : The model provides a common language for teams to discuss and measure quality.

Quality Standard Models – How it Works

- The ISO Quality Model is like a checklist for software quality. It asks:
  - Does it work? (Functionality)
  - Is it fast and efficient? (Performance)
  - Is it secure? (Security)
  - Is it easy to use? (Usability)

Quality Standard Models – ISOs

Aspect	ISO/IEC 9126	ISO/IEC 25010	ISO/IEC 9001
Focus	Software product quality	Software product and system quality	Organizational processes and quality management
Scope	Post-development evaluation	Entire software lifecycle	All industries, including software development
Key Characteristics	6 (Functionality, Reliability, Usability, etc.)	8 (Includes Security and Compatibility)	Process-driven (Customer focus, Leadership, etc.)
Purpose	Evaluate software quality	Evaluate and improve software quality	Ensure consistent quality through process management
Industry	Software-specific	Software-specific	General (applies to all industries)
Status	Replaced by ISO/IEC 25010	Current and updated	Current and widely used

Quality Standard Models – How it Works

Aspect	ISO/IEC 9126	ISO/IEC 25010	ISO/IEC 9001
Key Characteristics	1. Functionality 2. Reliability 3. Usability 4. Efficiency 5. Maintainability 6. Portability	1. Functionality 2. Performance 3. Compatibility 4. Usability 5. Reliability 6. Security 7. Maintainability 8. Portability	Emphasizes customer satisfaction , continuous improvement , and compliance with regulations.

Quality Standard Models – Six Sigma

- Six Sigma is a data-driven methodology aimed at improving quality by identifying and eliminating defects or errors in **processes**.
- It focuses on reducing variability and ensuring consistency in outputs, leading to near-perfect quality.
- The term "Six Sigma" refers to a statistical concept where a process produces no more than 3.4 defects per million opportunities (DPMO)

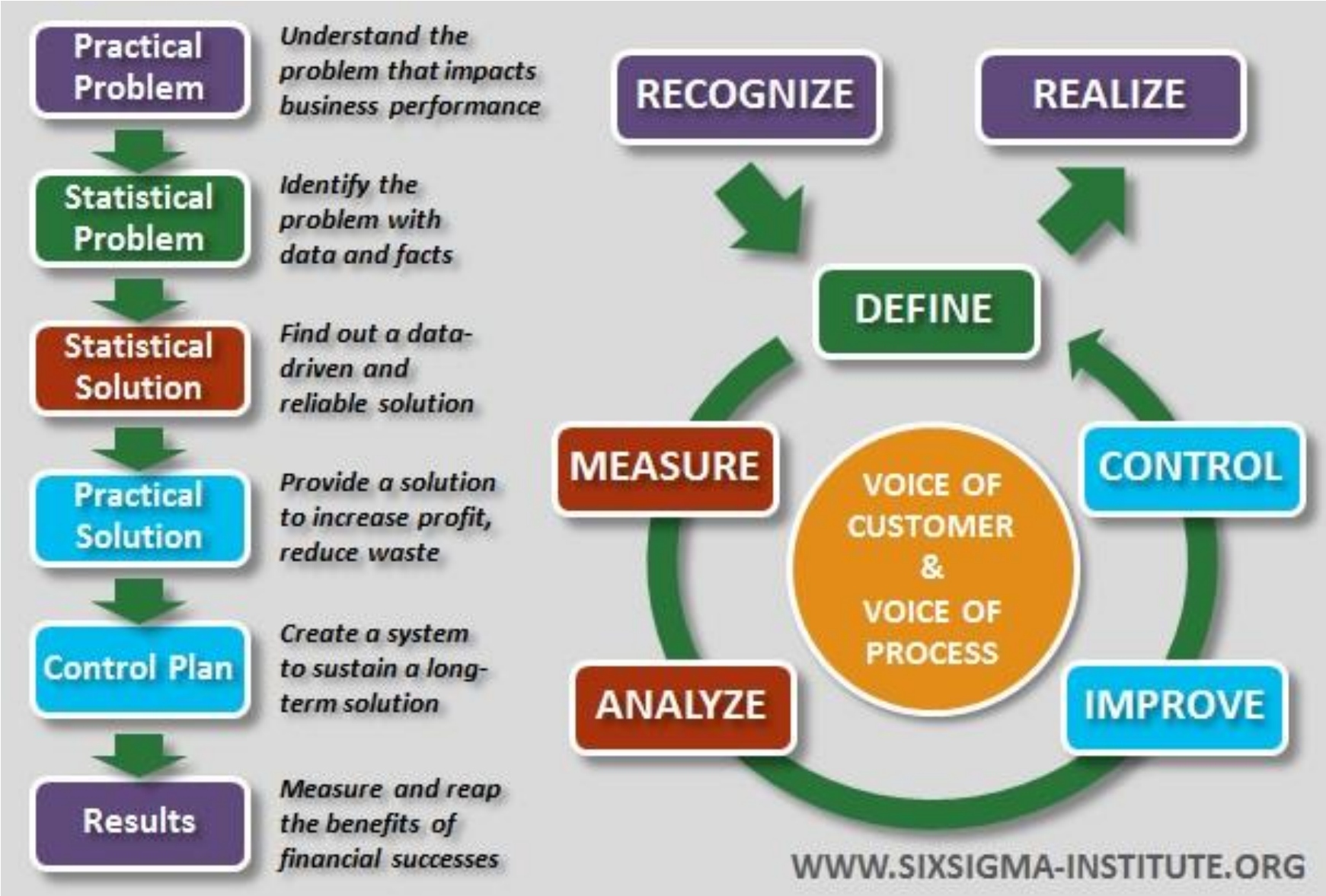
Quality Standard Models – Six Sigma

- **Six Sigma seeks to:**
  - Minimize defects and errors.
  - Improve process efficiency.
  - Enhance customer satisfaction by delivering consistent, high-quality products or services.

Quality Standard Models – Six Sigma - Principles

- Customer Focus : Prioritizes meeting customer expectations and requirements.
- Data-Driven Decisions : Uses statistical analysis to identify root causes of problems.
- Process Improvement : Focuses on optimizing processes to reduce variation and waste.
- Continuous Improvement : Encourages ongoing refinement of processes for better outcomes.

Quality Standard Models – Six Sigma - DMAIC Framework



Quality Standard Models – Six Sigma - DMAIC Framework

- Six Sigma uses the DMAIC methodology to improve existing processes:
- Define : Identify the problem and define project goals based on customer needs.
- Measure : Collect data to understand the current process performance.
- Analyze : Use statistical tools to identify root causes of defects or inefficiencies.
- Improve : Implement solutions to address root causes and optimize the process.
- Control : Monitor and sustain improvements to ensure long-term success.

Quality Standard Models – Six Sigma - Levels of Six Sigma

- Six Sigma professionals are categorized into "belts," similar to martial arts:

- Green Belt : Team members trained in Six Sigma tools and techniques.
- Black Belt : Experts who lead Six Sigma projects.
- Master Black Belt : Senior experts who mentor and guide Black Belts.
- Yellow Belt : Basic understanding of Six Sigma concepts, often supporting Green Belts.

Quality Standard Models – Six Sigma - Example

- A software development team uses Six Sigma to reduce bugs in their application:

- **Define** : Goal is to reduce post-release defects by 50%.
- **Measure** : Analyze defect rates in the last three releases.
- **Analyze** : Identify that poor code reviews are causing most defects.
- **Improve** : Implement stricter code review guidelines and automated testing.
- **Control** : Monitor defect rates in future releases to ensure improvement.

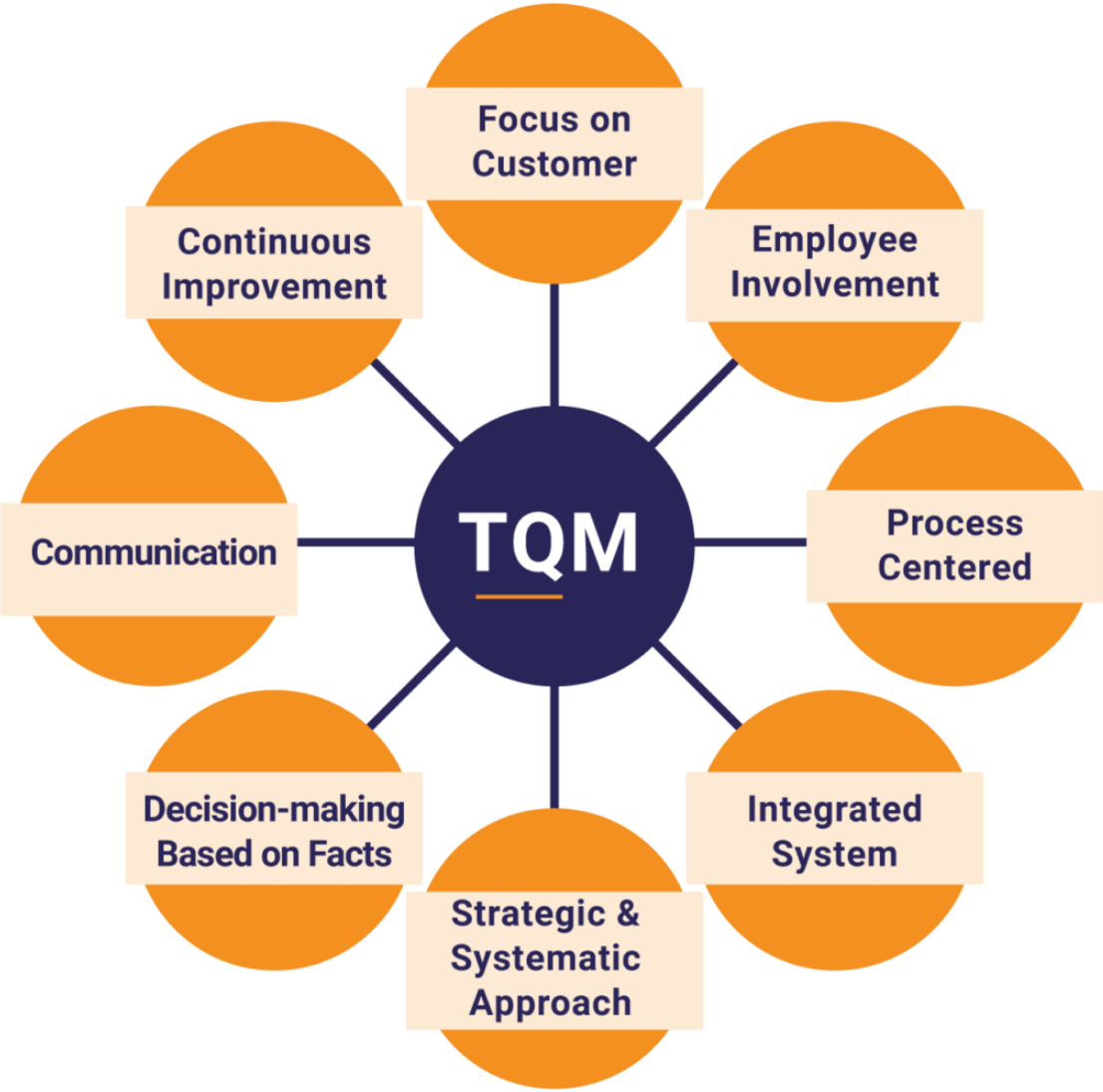
Quality Standard Models – TQM

- Total Quality Management (TQM) is a holistic approach to quality management that focuses on continuous improvement, customer satisfaction, and the involvement of all stakeholders in an organization.
- In software engineering , TQM is applied to ensure that software development processes and products meet or exceed customer expectations by fostering a culture of quality throughout the organization.

Quality Standard Models – TQM

- **TQM in software engineering emphasizes:**
- Customer-Centric Approach : Ensuring that the software meets or exceeds user requirements and expectations.
- Continuous Improvement : Regularly refining processes and practices to enhance efficiency and quality.
- Employee Involvement : Engaging all team members—developers, testers, managers, and stakeholders—in achieving quality goals.
- Process Orientation : Standardizing and optimizing workflows to reduce errors, inefficiencies, and rework.

Quality Standard Models – TQM



Quality Standard Models – TQM - Principle

- **Customer Focus**
- The primary goal of TQM is to deliver software that satisfies customer needs. This involves understanding user requirements, gathering feedback, and incorporating it into the development process.
- Example: Regularly conducting user surveys or usability testing to ensure the software aligns with user expectations.

Quality Standard Models – TQM - Principle

- **Continuous Improvement**
- TQM promotes a mindset of constant refinement. Teams analyze processes, identify bottlenecks, and implement improvements iteratively.
- Example: Using Agile methodologies to incorporate feedback and improve features in each sprint.

Quality Standard Models – TQM - Principle

- **Employee Empowerment**
- TQM encourages active participation from all team members. Developers, testers, and other stakeholders are empowered to suggest improvements and take ownership of quality.
- Example: Conducting brainstorming sessions where team members propose ideas to reduce defects or streamline workflows.

Quality Standard Models – TQM - Principle

- **Process-Centric Approach**
- TQM emphasizes defining, documenting, and standardizing processes to ensure consistency and repeatability in software development.
- Example: Implementing coding standards, automated testing frameworks, and code review processes to maintain high-quality outputs.

Quality Standard Models – TQM - Principle

- **Data-Driven Decision Making**
- Decisions in TQM are based on measurable data rather than assumptions. Metrics like defect rates, cycle times, and user feedback are analyzed to guide improvements.
- Example: Using tools like JIRA or SonarQube to track bugs and technical debt, then using this data to prioritize fixes.

Quality Standard Models – TQM - Principle

- Leadership Commitment
- Leadership plays a critical role in fostering a culture of quality. Managers must support TQM initiatives by providing resources, setting clear goals, and leading by example.
- Example: Senior management conducts regular reviews of quality metrics and allocates budgets for training and tools.

Quality Standard Models – TQM - Principle

- **Systematic Problem Solving**
- TQM uses structured methods like PDCA (Plan-Do-Check-Act) or Root Cause Analysis (RCA) to address issues systematically.
- Example: Applying RCA to identify why a specific bug occurred and implementing preventive measures to avoid recurrence.

Quality Standard Models – TQM - Principle

- **Long-Term Thinking**
- TQM focuses on sustainable improvements rather than short-term fixes. It aims to build robust processes that consistently deliver high-quality software over time.
- Example: Investing in automation tools for testing and deployment to improve long-term efficiency.

Quality Standard Models – TQM - Implementation

1. Requirement Gathering and Analysis	• Work closely with customers to understand their needs and expectations. • Use techniques like prototyping or user stories to clarify requirements and ensure alignment.
2. Design and Development	• Follow standardized design principles and coding guidelines. • Use tools like version control systems (e.g., Git) and code review platforms to maintain consistency.
3. Testing and Quality Assurance	• Implement rigorous testing processes, including unit testing, integration testing, and user acceptance testing (UAT). • Use automated testing tools to ensure comprehensive coverage and faster feedback loops.
4. Deployment and Maintenance	• Monitor software performance in production using tools like application performance monitoring (APM). • Continuously gather user feedback and release updates to address issues or add new features.
5. Feedback Loop	• Collect feedback from users, stakeholders, and team members after each release. • Use this feedback to refine processes and improve future releases.

Quality Standard Models – TQM - Challenges

- **Cultural Resistance**
  - Employees may resist changes to established workflows or struggle to adapt to a quality-focused mindset.
- **Resource Constraints**
  - Implementing TQM requires investment in tools, training, and process redesign, which may strain budgets.
- **Measurement Complexity**
  - Defining and tracking meaningful quality metrics can be challenging, especially in complex software projects.
- **Time-Consuming**
  - TQM is a long-term strategy, and results may not be immediately visible, which can test organizational patience.

Quality Standard Models – TQM - Example

- A software company adopts TQM to improve its product quality:
- **Customer Focus** : Conducts user interviews to identify pain points in the current software.
- **Continuous Improvement** : Implements Agile practices to release incremental updates based on feedback.
- **Employee Involvement** : Holds weekly meetings where developers and testers discuss ways to reduce bugs.
- **Process Standardization** : Introduces coding standards and mandatory peer reviews to catch issues early.
- **Data-Driven Decisions** : Tracks defect rates and user satisfaction scores to measure progress.
- **Outcome** : Over time, the company reduces post-release defects by 40% and improves customer retention.

Capability Maturity Model (CMM)

- A framework designed to assess and improve the maturity of processes within an organization, particularly in software development.
- Maturity of Process - It measures the extent to which a process is structured, standardized, and capable of delivering consistent and efficient results.
- Clear documentation, effective controls, and continuous improvement mechanisms

Capability Maturity Model (CMM) – Five Levels

- Initial: Processes are ad hoc and chaotic, with unpredictable outcomes.
- Repeatable: Basic project management processes are established, enabling repeatable results.
- Defined: Processes are well-documented, standardized, and integrated across the organization.
- Managed: Processes are measured and controlled using quantitative data.
- Optimizing: Continuous process improvement is emphasized through innovation and feedback.

Capability Maturity Model (CMM) – Application

- **Process Evaluation:**
  - CMM allows organizations to assess their existing software development processes and identify areas that need improvement.
  - It helps create a structured approach to manage projects effectively.

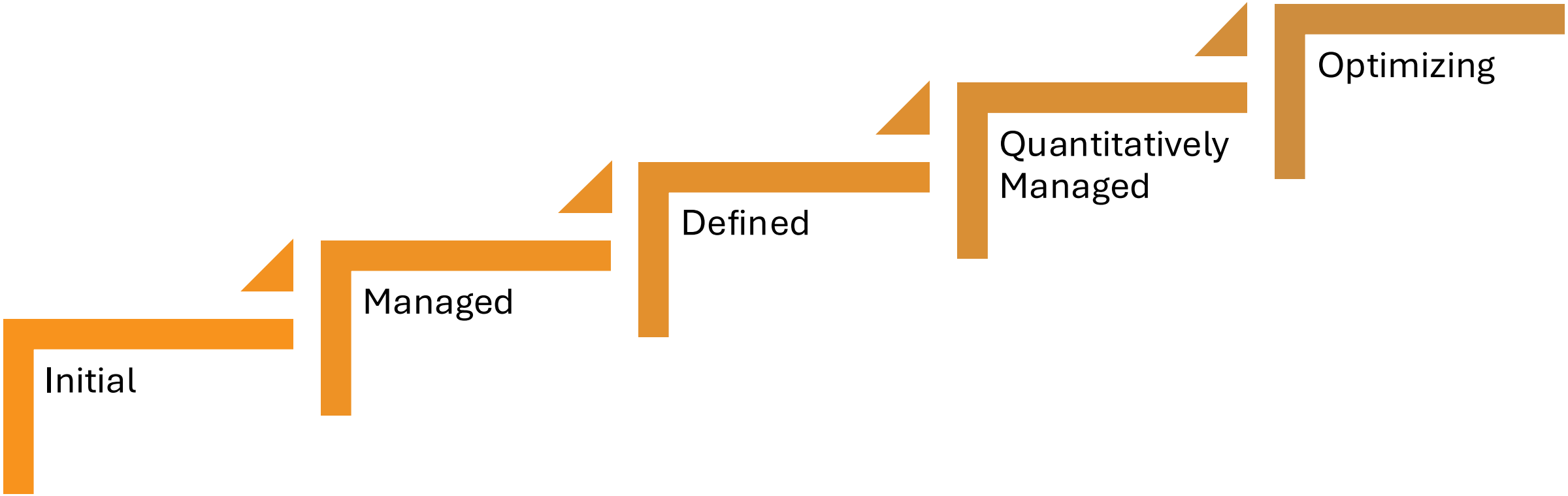
Capability Maturity Model (CMM) – Application

- **Enhanced Project Management:**
  - The model ensures consistent project execution, reducing risks and improving delivery timelines.
  - At higher maturity levels, quantitative metrics are used to monitor and control project performance.

Capability Maturity Model (CMM) – Application

- **Quality Improvement:**
  - By standardizing processes, CMM improves the overall quality of software products, minimizing defects and ensuring reliability.
- **Optimized Resource Utilization:**
  - Processes at advanced maturity levels are optimized, leading to better utilization of resources—whether human, technical, or financial.
- **Continuous Improvement:**
  - At the optimizing level, organizations focus on innovation and continuous improvement, keeping up with evolving industry standards and customer needs.

Capability Maturity Model (CMM) –



Capability Maturity Model Integration (CMMI) – Levels

Maturity Levels

Capability Levels

Capability Maturity Model Integration (CMMI) –

Aspect	CMM	CMMI
Scope	Focuses only on **software process improvement**	Covers **software, systems, hardware, and services**
Model Type	**Prescriptive** – Defines rigid levels and processes	**Flexible** – Allows customization and process tailoring
Structure	Follows **staged levels (1-5 maturity levels)**	Supports both **staged (maturity levels) and continuous (capability levels) representations**
Improvement Approach	Focused on individual processes	**Integrated approach**, addressing multiple disciplines together
Adoption	**Older model**, now largely replaced	**Current industry standard**, widely adopted

Capability Levels–

Levels	Description
0 - Incomplete	The process is not performed or only partially performed.
1 – Performed	The process is executed but may be unpredictable or inconsistent.
2 – Managed	The process is planned, monitored, and controlled. It follows policies and is well-managed.
3 – Defined	The process is well-documented and tailored from organizational standards. Best practices are followed.

Maturity Levels–

Levels	Description
1 - Initial	Processes are ad hoc, chaotic, or unpredictable. No stable environment for success.
2 - Managed	Basic project management is established. Processes are repeatable but may vary across teams.
3 - Defined	Organization-wide standard processes are established and used consistently.
4 - Quantitatively Managed	Processes are measured and controlled using quantitative techniques.
5 - Optimizing	Continuous process improvement is implemented using innovation and feedback.

Quality Assurance (QA)

- QA is a proactive, process-oriented approach that focuses on preventing defects in the process used to make a product or deliver a service.

- **Key Characteristics**
  - Ensures process quality
  - Prevents issues before they occur
  - Involves standards, procedures, and guidelines
  - Focuses on continuous improvement
  - Typically includes audits, training, process checklists, and reviews

Quality Assurance (QA)

- QA includes
  - writing coding standards,
  - conducting code reviews, and
  - defining testing protocols. Etc.

Quality Control (QC)

- QC is a reactive, product-oriented approach that focuses on identifying defects in the final product.

- **Key Characteristics**
  - Ensures product quality
  - Detects issues after they occur
  - Involves testing, inspection, and validation
  - Focuses on meeting quality specifications
  - Usually involves manual/automated testing

Quality Control (QC)

- QC would involve
  - Executing test cases,
  - Finding bugs, and
  - Reporting issues before a release.

Quality Control (QC) Vs Quality Assurance (QA)

Aspect	QA	QC
Focus	Process	Product
Objective	Prevent Defects	Detect Defects
Type	Proactive	Reactive
Performed by	Process/QA team	Testers/Inspectors
Techniques	Process audits, documentation	Testing (manual, automation), inspection
Outcome	Improved process	Verified product
	Doing things the right way	Checking the Final Product

Quality Management

- The process of ensuring that products or services meet consistent standards and satisfy customer expectations.

- In simple terms, it's about making things right, keeping them right, and improving them over time.

Quality Management = Plan it right (Planning) + Do it right (QA) + Check it right (QC) + Keep getting better (Improvement)

Quality Management – Four Parts

Quality Planning

- Deciding **what "quality" means** for the project/product.
- Setting **goals, standards, and procedures** before work begins.
- Example: Defining performance benchmarks, security standards, or coding guidelines in a software project.

Quality Assurance

- **Creating and following processes** to avoid mistakes.
- Focuses on **how the work is done**.
- Example: Having code reviews or CI/CD pipelines to maintain code quality.

Quality Control

- **Checking the final product** to find and fix defects.
- Focuses on **what is produced**.
- Example: Testing a software app to find bugs before release.

Continuous Improvement

- **Learning from mistakes** and improving processes.
- Common models: **PDCA (Plan-Do-Check-Act)**, **Six Sigma**, **Kaizen**.
- Example: Collecting feedback from users after product launch and using it to improve the next version.

Quality Management – Key Methods

- PDCA Cycle (Plan–Do–Check–Act)
- Six Sigma
- Statistical Process Control (SPC)
- Total Quality Management (TQM)
- ISO 9001 (Quality Standards)
- Failure Mode and Effects Analysis (FMEA)

Quality Management – Key Methods

Method	Focus	Key Benefit
TQM	Whole organization	Everyone improves quality
PDCA	Iterative improvement	Simple and flexible
Six Sigma	Reducing defects	Data-driven accuracy
SPC	Monitoring processes	Early detection of issues
FMEA	Risk analysis	Prevents major failures
ISO 9001	Compliance	Trusted industry standard
Kaizen	Small improvements	Continuous team learning

Quality Factors

- Quality Factors refer to the key characteristics that define how good a software product is.
- These are used to evaluate, measure, and ensure software quality from both a technical and user perspective.

Quality Factors

Quality Factor	What it Means
Functionality	Meets user needs correctly
Performance	Fast and resource-efficient
Reliability	Works consistently without crashing
Usability	Easy to use and understand
Maintainability	Easy to fix and update
Portability	Can run on different systems
Security	Safe from threats and data leaks
Testability	Easy to test for bugs and errors

Quality Factors

- Functionality
  - Does the software do what it's supposed to do?
  - Includes:
    - Correctness: Produces the right results
    - Security: Protects against unauthorized access
    - Interoperability: Works well with other systems

Quality Factors

- Performance Efficiency
  - How well does the software use resources?
- Includes:
  - Response Time
  - Throughput
  - Resource Usage

Quality Factors

- Reliability
  - Can the software perform well under different conditions?
- Includes:
  - Fault Tolerance: Works even when some parts fail
  - Availability: Ready when needed
  - Recoverability: Recovers from crashes

Quality Factors

- Usability
  - How easy is it for users to learn and use the software?
- Includes:
  - Learnability
  - Operability
  - User Error Protection

Quality Factors

- Maintainability
  - How easy is it to fix, update, or enhance the software?
- Includes:
  - Modularity
  - Reusability
  - Analyzability

Quality Factors

- Portability
  - Can the software run on different platforms?
- Includes:
  - Adaptability
  - Installability
  - Replaceability

Quality Factors

- Security (Also a sub-factor of functionality in some models)
  - Protects data and resists attacks
- Includes:
  - Confidentiality
  - Integrity
  - Authentication

Quality Factors by McCalls

- Product Operation = Correctness, Reliability, Efficiency, Integrity, Usability

- Product Revision =Maintainability, Flexibility, Testability

- Product Transition = Portability, Reusability, Interoperability

Quality Factors by McCalls

Quality Factor	Description
Product Operations	
Correctness	Does it meet user requirements and perform desired functions?
Reliability	Does it perform consistently without failure?
Efficiency	Does it use system resources like memory and CPU wisely?
Integrity	Is it secure against unauthorized access and data corruption?
Usability	Is it user-friendly and easy to understand?
Product Revision	
Maintainability	How easy is it to fix bugs or make changes?
Flexibility	Can it adapt to new environments or requirements?
Testability	How easy is it to test the software and find bugs?
Product Transition	
Portability	Can it run on different hardware or OS platforms?
Reusability	Can parts of the code be used in other projects?
Interoperability	Can it work smoothly with other systems/software?